



ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ

Автоматическое дифференцирование

МЕТОДЫ ОПТИМИЗАЦИИ

НЕДЕЛЯ 3

План лекции

1. Зачем считать градиенты и чем плохи конечные разности
2. Вычислительный граф
3. Пример: логистическая регрессия
4. Обратный режим AD (reverse, VJP)
5. Прямой режим AD (forward, JVP)
6. Выбор режима, память и чек-пойнтинг

Задача



Хотим решать

$$L(\mathbf{w}) \rightarrow \min_{\mathbf{w} \in \mathbb{R}^d}.$$

- Такие задачи возникают при обучении моделей: $\mathbf{w}$ — вектор параметров, d — число параметров.

Модель	Всего	Активных (MoE)
Llama 4 Maverick ¹	400 млрд	17 млрд
DeepSeek-V3 ²	671 млрд	37 млрд
Kimi K2 ³	1 трлн	32 млрд
Qwen3.8 ⁴	2,4 трлн	95 млрд
GPT-5, Claude, Gemini, Grok	не публикуют	—

У MoE в памяти лежат все d параметров, на токен активна только часть.

¹ Meta AI. *Llama 4 Model Card*, 2025. github.com/meta-llama/llama-models.

² DeepSeek-AI. *DeepSeek-V3 Technical Report*, 2024. github.com/deepseek-ai/DeepSeek-V3.

³ Moonshot AI. *Kimi K2: Open Agentic Intelligence*, 2025. [arXiv:2507.20534](https://arxiv.org/abs/2507.20534).

⁴ Qwen Team, Alibaba Cloud. *Qwen3.8-2.4T-A95B Model Card*, 2025.

Три способа взять производную

- Методы нулевого порядка (только оракул L) плохо масштабируются.
- Методы первого порядка используют $L'(\mathbf{w})$.
- Методы второго порядка требуют гессиан $\mathbf{H}$ или произведения $\mathbf{H}\mathbf{v}$.

	Численно	Символьно	AD
Идея	конечные разности	правила на формулах	правила на графе
Точность	$\sim \varepsilon_{\text{mach}}$	точная	точная
Стоимость L'	$\mathcal{O}(dT)$	взрыв выражений	$\mathcal{O}(T)$ (reverse)
Пример	<code>scipy.approx_fprime</code>	SymPy, Mathematica	torch, jax

T — стоимость одного вычисления $L(\mathbf{w})$.

- AD применяет формулу производной сложной функции.
- Но к *программе* вычисления L , а не к аналитической формуле.

Конечные разности

Для каждой координаты

$$\frac{\partial L}{\partial w_k}(\mathbf{w}) \approx \frac{L(\mathbf{w} + \varepsilon \mathbf{e}_k) - L(\mathbf{w})}{\varepsilon}, \quad \mathbf{e}_k = (0, \dots, \underset{k}{1}, \dots, 0).$$

Центральная разность точнее:

$$\frac{\partial L}{\partial w_k}(\mathbf{w}) \approx \frac{L(\mathbf{w} + \varepsilon \mathbf{e}_k) - L(\mathbf{w} - \varepsilon \mathbf{e}_k)}{2\varepsilon}.$$

- **Q:** если одно вычисление L стоит T , сколько стоит L' ?
- Ответ: $\sim dT$ (или $2dT$ для центральной схемы).
- Кроме того, надо выбирать ε : слишком маленькое значение — шум округления, слишком большое значение — ошибка аппроксимации.

Проклятие размерности для методов нулевого порядка

Градиентный спуск vs оценка градиента G по значениям L :

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha_k L'(\mathbf{w}_k) \quad \text{vs} \quad \mathbf{w}_{k+1} = \mathbf{w}_k - \alpha_k G.$$

	Гладкая выпуклая	Гладкая сильно выпуклая
GD ¹	$L(\mathbf{w}_k) - L^* \sim \mathcal{O}(1/k)$	$\ \mathbf{w}_k - \mathbf{w}^*\ ^2 \sim \mathcal{O}((1 - \mu/L)^k)$
zero-order GD ²	$\sim \mathcal{O}(d/k)$	$\sim \mathcal{O}((1 - \mu/(dL))^k)$

Для двухточечных оценок зависимость от d не убрать: без градиента большие задачи решать дорого.

¹ Y. Nesterov. *Lectures on Convex Optimization*. Springer, 2018. fmin.xyz.

² J. C. Duchi, M. I. Jordan, M. J. Wainwright, A. Wibisono. *Optimal rates for zero-order convex optimization: the power of two function evaluations*. arXiv:1312.2139, 2015.

Теорема: градиент за $\mathcal{O}(T)$

Теорема

Если вычисление $L(\mathbf{w})$ стоит T , то существует алгоритм, который считает $L'(\mathbf{w})$ за $\mathcal{O}(T)$ ¹.

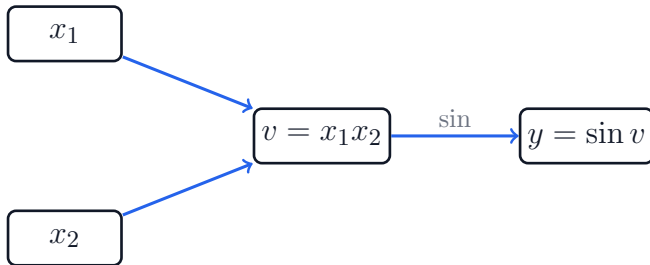
- Это *обратный* режим автоматического дифференцирования.
- Константа небольшая: полный градиент стоит как несколько вызовов L .
- Именно поэтому обучают сети с миллиардами параметров.
- Дальше: как устроен этот алгоритм и когда нужен прямой режим.

¹ S. Linnainmaa. *Taylor expansion of the accumulated rounding error*. BIT, 16(2):146–160, 1976.

Вычислительный граф



- Программа L — композиция элементарных операций $(+, \times, \sin, \exp, \log, \dots)$.
- У каждой операции известна производная.
- Вершины — промежуточные значения v_i .
- Рёбра — зависимости «вход $\rightarrow$ выход».
- Тот же граф, что на прошлой лекции: $y = \sin(x_1 x_2)$.



AD = вычисление производной по этому графу по правилу сложной функции, а не по аналитической формуле $y(x_1, x_2)$.

Связь с прошлой лекцией

$h(\mathbf{x}) = f(g(\mathbf{x}))$, $g: \mathbb{R}^n \rightarrow \mathbb{R}^m$, $f: \mathbb{R}^m \rightarrow \mathbb{R}$, $\mathbf{y} = g(\mathbf{x})$. Покоординатно:

$$\frac{\partial h}{\partial x_j} = \sum_{i=1}^m \frac{\partial f}{\partial y_i}(\mathbf{y}) \cdot \frac{\partial g_i}{\partial x_j}(\mathbf{x}), \quad j = 1, \dots, n, \quad (J_g)_{ij} = \frac{\partial g_i}{\partial x_j}.$$

Собираем координаты в вектор:

$$\boxed{\underbrace{h'(\mathbf{x})}_{n \times 1} = \underbrace{J_g(\mathbf{x})^\top}_{n \times m} \underbrace{f'(g(\mathbf{x}))}_{m \times 1}}$$

Связь с прошлой лекцией: K функций

1. Композиция. $\mathbf{v}_0 = \mathbf{x} \in \mathbb{R}^{n_0}$, $\mathbf{v}_i = f_i(\mathbf{v}_{i-1})$, $f_i: \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}^{n_i}$, $i = 1, \dots, K$. Итог: $\mathbf{v}_K \in \mathbb{R}^{n_K}$.

2. Полный якобиан. Локальные $J_i = \partial \mathbf{v}_i / \partial \mathbf{v}_{i-1} \in \mathbb{R}^{n_i \times n_{i-1}}$:

$$\underbrace{J}_{n_K \times n_0} = \underbrace{J_K}_{n_K \times n_{K-1}} \underbrace{J_{K-1}}_{n_{K-1} \times n_{K-2}} \cdots \underbrace{J_1}_{n_1 \times n_0}.$$

3. Считать J по столбцам или по строкам.

По столбцу j : $J \mathbf{e}_j = J_K(J_{K-1}(\cdots(J_1 \mathbf{e}_j)))$ forward

По строке i : $\mathbf{e}_i^\top J = J^\top \mathbf{e}_i = J_1^\top(J_2^\top(\cdots(J_K^\top \mathbf{e}_i)))$ reverse или backward

На каждом ребре f_i , помимо вычисления самой функции, нужно вычислить произведения входного вектора с J_i :

$$\text{JVP}(f_i, \mathbf{v}_{i-1}, \mathbf{u}) = J_i \mathbf{u}, \quad \text{VJP}(f_i, \mathbf{v}_{i-1}, \mathbf{w}) = J_i^\top \mathbf{w}.$$

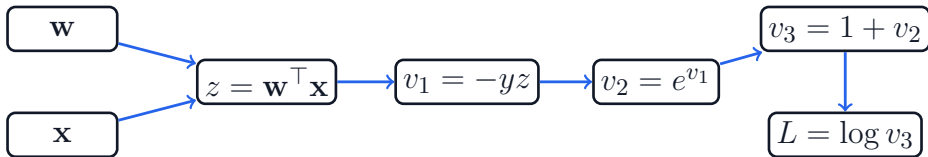
Дальше — как считать JVP/VJP на вершинах вычислительного графа.

Логистический лосс (один объект)

$\mathbf{x} \in \mathbb{R}^d$, $y \in \{\pm 1\}$ фиксированы; $\mathbf{w} \in \mathbb{R}^d$ — параметры. Цель: $L'(\mathbf{w})$.

$$L(\mathbf{w}) = \log(1 + \exp(-y \mathbf{w}^\top \mathbf{x})).$$

Граф: $z = \mathbf{w}^\top \mathbf{x}$, $v_1 = -yz$, $v_2 = e^{v_1}$, $v_3 = 1 + v_2$, $L = \log v_3$;



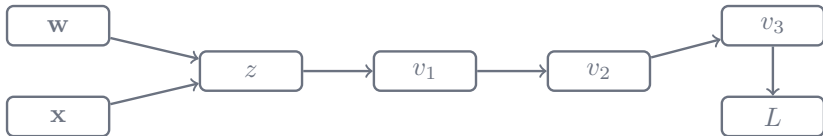
Дальше:

- reverse — весь L' за один обратный проход
- forward — по одной координате за проход

Reverse: шаг за шагом

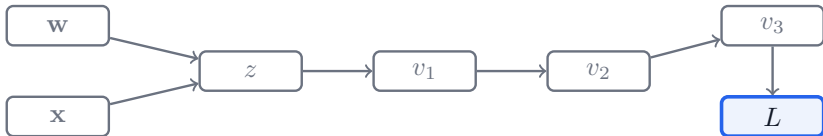


Прямой проход выполнен, сохранены z , v_1 , v_2 , v_3 , L . Старт обратного: $\bar{L} = 1$.



Reverse: шаг за шагом

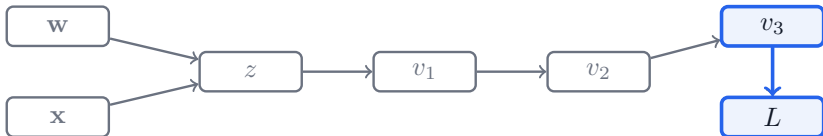
Прямой проход выполнен, сохранены z , v_1 , v_2 , v_3 , L . Старт обратного: $\bar{L} = 1$.



$$\bar{L} = 1$$

Reverse: шаг за шагом

Прямой проход выполнен, сохранены z , v_1 , v_2 , v_3 , L . Старт обратного: $\bar{L} = 1$.

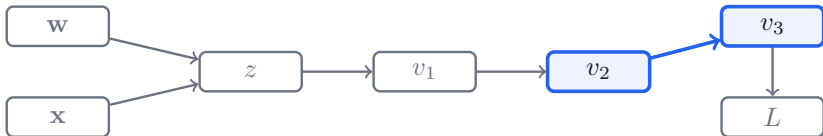


$$\bar{L} = 1$$

$$L = \log v_3: \bar{v}_3 = \bar{L} \cdot 1/v_3 = 1/v_3$$

Reverse: шаг за шагом

Прямой проход выполнен, сохранены z , v_1 , v_2 , v_3 , L . Старт обратного: $\bar{L} = 1$.



$$\bar{L} = 1$$

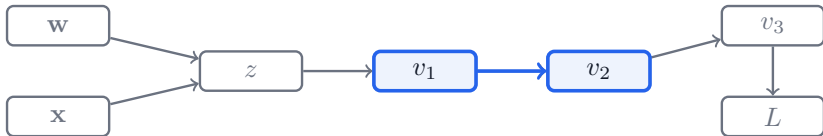
$$L = \log v_3: \bar{v}_3 = \bar{L} \cdot 1/v_3 = 1/v_3$$

$$v_3 = 1 + v_2: \bar{v}_2 = \bar{v}_3 \cdot 1 = 1/v_3$$

Reverse: шаг за шагом



Прямой проход выполнен, сохранены z , v_1 , v_2 , v_3 , L . Старт обратного: $\bar{L} = 1$.



$$\bar{L} = 1$$

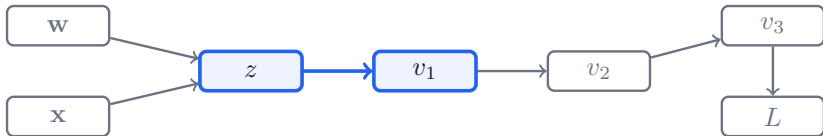
$$L = \log v_3: \bar{v}_3 = \bar{L} \cdot 1/v_3 = 1/v_3$$

$$v_3 = 1 + v_2: \bar{v}_2 = \bar{v}_3 \cdot 1 = 1/v_3$$

$$v_2 = e^{v_1}: \bar{v}_1 = \bar{v}_2 \cdot e^{v_1} = v_2/v_3$$

Reverse: шаг за шагом

Прямой проход выполнен, сохранены z , v_1 , v_2 , v_3 , L . Старт обратного: $\bar{L} = 1$.



$$\bar{L} = 1$$

$$L = \log v_3: \bar{v}_3 = \bar{L} \cdot 1/v_3 = 1/v_3$$

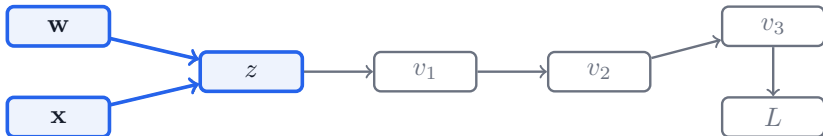
$$v_3 = 1 + v_2: \bar{v}_2 = \bar{v}_3 \cdot 1 = 1/v_3$$

$$v_2 = e^{v_1}: \bar{v}_1 = \bar{v}_2 \cdot e^{v_1} = v_2/v_3$$

$$v_1 = -yz: \bar{z} = \bar{v}_1 \cdot (-y) = -y v_2/v_3$$

Reverse: шаг за шагом

Прямой проход выполнен, сохранены z , v_1 , v_2 , v_3 , L . Старт обратного: $\bar{L} = 1$.



$$\bar{L} = 1$$

$$L = \log v_3: \bar{v}_3 = \bar{L} \cdot 1/v_3 = 1/v_3$$

$$v_3 = 1 + v_2: \bar{v}_2 = \bar{v}_3 \cdot 1 = 1/v_3$$

$$v_2 = e^{v_1}: \bar{v}_1 = \bar{v}_2 \cdot e^{v_1} = v_2/v_3$$

$$v_1 = -yz: \bar{z} = \bar{v}_1 \cdot (-y) = -y v_2/v_3$$

$$z = \mathbf{w}^\top \mathbf{x}: L'(\mathbf{w}) = \bar{z} \mathbf{x} = -y \frac{v_2}{v_3} \mathbf{x}$$

Анализ сложности: reverse

Пусть вычисление скалярного L стоит $T = \text{OPS}\{L\}$, число входов $n = \dim \mathbf{w}$.

Reverse (общий случай)	
Вычисления	$\text{OPS}\{L'\} \leq \omega T, \omega \leq 5^1$; полный градиент за <i>один</i> обратный проход
Память	$\Theta(\#\{v_i\})$ — промежуточные значения прямого прохода (лента / активации)

На логистической функции потерь:

- сложность градиента $\Theta(d)$ сравнима со сложностью L : $\Theta(d) = T$ — частный случай ωT
- промежуточных вершин $O(1)$ (z, v_1, v_2, v_3) $\Rightarrow$ память $O(1)$ сверх $\mathbf{w}, \mathbf{x}$
- в глубоких сетях активаций много — память ленты становится узким местом reverse
- в torch: метод `backward()` считает градиенты у листьев с `requires_grad=True` именно таким образом

Чек-пойнтинг

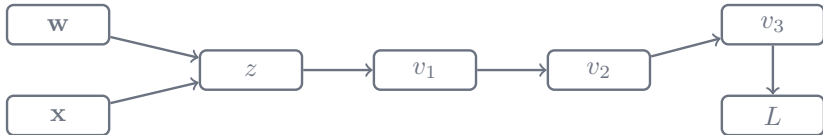
Сохраняем только часть вершин; остальные пересчитываем при обратном проходе. Память падает ценой дополнительных вычислений. Стандартные реализации: `torch.utils.checkpoint`, `jax.checkpoint`.

¹ A. Griewank, A. Walther. *Evaluating Derivatives*. SIAM, 2008.

Forward: шаг за шагом



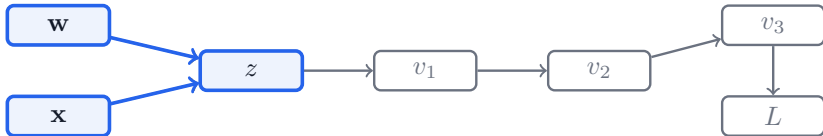
Тот же лосс. Считаем $\partial L / \partial w_k$: несём значения v_i и тангенсы $\dot{v}_i$ *вперёд*. Старт: $\dot{w}_k = 1$, $\dot{w}_\ell = 0$ ($\ell \neq k$), $\dot{x} = 0$.



Forward: шаг за шагом



Тот же лосс. Считаем $\partial L / \partial w_k$: несём значения v_i и тангенсы $\dot{v}_i$ *вперёд*. Старт: $\dot{w}_k = 1$, $\dot{w}_\ell = 0$ ($\ell \neq k$), $\dot{x} = 0$.

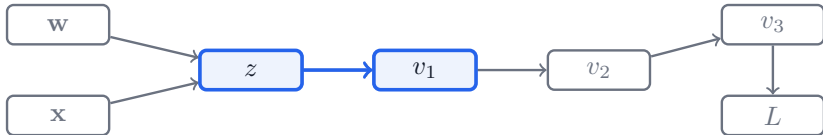


$$z = \mathbf{w}^\top \mathbf{x}: \dot{z} = x_k \dot{w}_k = x_k$$

Forward: шаг за шагом



Тот же лосс. Считаем $\partial L / \partial w_k$: несём значения v_i и тангенсы $\dot{v}_i$ *вперёд*. Старт: $\dot{w}_k = 1$, $\dot{w}_\ell = 0$ ($\ell \neq k$), $\dot{x} = 0$.

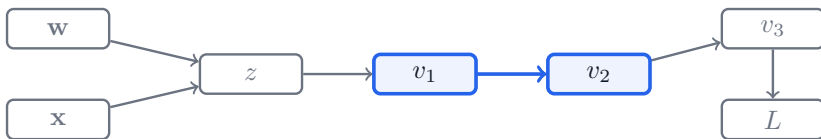


$$z = \mathbf{w}^\top \mathbf{x}: \dot{z} = x_k \dot{w}_k = x_k$$

$$v_1 = -yz: \dot{v}_1 = (-y) \dot{z} = -y x_k$$

Forward: шаг за шагом

Тот же лосс. Считаем $\partial L / \partial w_k$: несём значения v_i и тангенсы $\dot{v}_i$ *вперёд*. Старт: $\dot{w}_k = 1$, $\dot{w}_\ell = 0$ ($\ell \neq k$), $\dot{x} = 0$.



$$z = \mathbf{w}^\top \mathbf{x}: \dot{z} = x_k \dot{w}_k = x_k$$

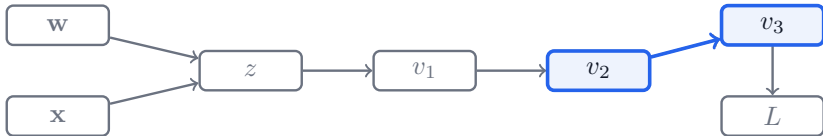
$$v_1 = -yz: \dot{v}_1 = (-y) \dot{z} = -yx_k$$

$$v_2 = e^{v_1}: \dot{v}_2 = e^{v_1} \dot{v}_1 = v_2(-yx_k)$$

Forward: шаг за шагом



Тот же лосс. Считаем $\partial L / \partial w_k$: несём значения v_i и тангенсы $\dot{v}_i$ *вперёд*. Старт: $\dot{w}_k = 1$, $\dot{w}_\ell = 0$ ($\ell \neq k$), $\dot{x} = 0$.



$$z = \mathbf{w}^\top \mathbf{x}: \dot{z} = x_k \dot{w}_k = x_k$$

$$v_1 = -yz: \dot{v}_1 = (-y) \dot{z} = -yx_k$$

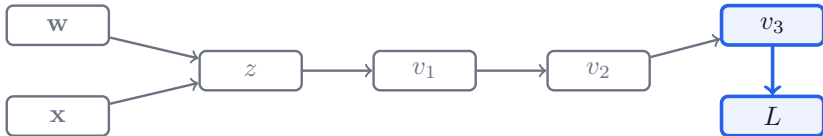
$$v_2 = e^{v_1}: \dot{v}_2 = e^{v_1} \dot{v}_1 = v_2(-yx_k)$$

$$v_3 = 1 + v_2: \dot{v}_3 = 1 \dot{v}_2 = -yx_k v_2$$

Forward: шаг за шагом



Тот же лосс. Считаем $\partial L / \partial w_k$: несём значения v_i и тангенсы $\dot{v}_i$ *вперёд*. Старт: $\dot{w}_k = 1$, $\dot{w}_\ell = 0$ ($\ell \neq k$), $\dot{x} = 0$.



$$z = \mathbf{w}^\top \mathbf{x}: \dot{z} = x_k \dot{w}_k = x_k$$

$$v_1 = -yz: \dot{v}_1 = (-y) \dot{z} = -yx_k$$

$$v_2 = e^{v_1}: \dot{v}_2 = e^{v_1} \dot{v}_1 = v_2(-yx_k)$$

$$v_3 = 1 + v_2: \dot{v}_3 = 1 \dot{v}_2 = -yx_k v_2$$

$$L = \log v_3: \dot{L} = 1/v_3 \dot{v}_3 = -yx_k \frac{v_2}{v_3} = \partial L / \partial w_k$$

Анализ сложности: forward



Пусть $L: \mathbb{R}^n \rightarrow \mathbb{R}$ стоит $T = \text{OPS}\{L\}$.

Forward (общий случай)	
Вычисления	один JVP: $\text{OPS}\{\langle L', \mathbf{u} \rangle\} \leq \omega_f T$, $\omega_f \leq 5^1$; полный L' : n проходов $\Rightarrow \mathcal{O}(nT)$
Память	как у вычисления L (текущие $v_i, \dot{v}_i$); промежуточные значения прямого прохода не нужны

На логистической функции потерь:

- $n = d$, $T = \Theta(d)$; один проход $\Theta(d) = T$ даёт $\partial L / \partial w_k$
- полный градиент: d проходов $\Rightarrow \Theta(dT) = \Theta(d^2)$
- память $\mathcal{O}(1)$ сверх входов — согласуется с общим правилом

¹ A. Griewank, A. Walther. *Evaluating Derivatives*. SIAM, 2008.

Когда какой режим

Нужен якобиан $J \in \mathbb{R}^{m \times n}$.

	Forward	Reverse
Один проход даёт	столбец J (JVP)	строку J (VJP)
Полный J	$\mathcal{O}(n T)$	$\mathcal{O}(m T)$
Память	мало	прямой проход
Типичный случай	$n \ll m$	$m \ll n$

- Нейросеть: $n \sim 10^6$ – 10^{12} , $m = 1$ — только reverse
- Много выходов, мало параметров — forward
- Смешанный режим: режут граф и комбинируют JVP/VJP

Вычислить произведение гессиана на вектор без хранения гессиана

Гессиан $f''(\mathbf{x}) \in \mathbb{R}^{n \times n}$ при большом n не хранят. Нужно произведение

$$f''(\mathbf{x}) \mathbf{v} = \nabla_{\mathbf{x}} \langle f'(\mathbf{x}), \mathbf{v} \rangle.$$

- Сначала reverse (или JVP) даёт градиент $f'(\mathbf{x})$.
- Скаляр $g(\mathbf{x}) = \langle f'(\mathbf{x}), \mathbf{v} \rangle$.
- Ещё один reverse по g даёт $\mathbf{H}\mathbf{v}$.
- Стоимость $\approx$ вычисление двух градиентов, а не $\mathcal{O}(n^2)$.

Вспомним про это свойство при обсуждении метода Ньютона и квазиньютоновских методов.

Чем AD не является

- Не конечные разности: нет ε , результат точный (с точностью FP).
- Не символьное дифференцирование: нет гигантской формулы $L'(\mathbf{w})$.
- Не «просто правило цепи»: это *эффективная раскладка* правила цепи по графу (JVP или VJP).
- Reverse дешёв по времени и устойчив численно.
- Reverse дорог по памяти — отсюда чек-пойнтинг.

Сравнение способов дифференцирования

	FD	Символьное	Forward	Reverse
Точность	низкая	точная	точная	точная
Градиент	$\mathcal{O}(dT)$	взрыв	$\mathcal{O}(dT)$	$\mathcal{O}(T)$
Память	мало	формулы	мало	прямой проход
Устойчивость	нет	да	да	да

Практика: `torch.autograd` и `jax.grad` — reverse; `jax.jvp` — forward; HVP — композиция двух проходов.

Что запомнить

- Конечные разности масштабируются как $\mathcal{O}(dT)$ и неустойчивы.
- AD считает производные по вычислительному графу элементарных операций.
- Forward/JVP: один столбец якобиана за проход.
- Reverse/VJP: одна строка (весь градиент скалярной L) за проход.
- Для обучения моделей — reverse; память лечат чек-пойнтингом.
- ∇v считается через AD без сборки гессиана.
- Тьюториал HIPS Autograd: github.com/HIPS/autograd.
- Мини-реализация AD: github.com/mattjj/autodidact.

Дальше



- Выпуклые множества
- Выпуклые функции
- Выпуклые задачи оптимизации
- Правила построения выпуклых функций

